

Technical Assessment Report: AI-Driven Network Traffic Analysis & Intrusion Detection System

Candidate: Vinodh Kumar

Role Applied: Cybersecurity and Network Security Research Intern

Organization: Megaminds IT Services

Assessment Mode: Round 1 — Take-Home Assignment

GitHub Repository: <https://github.com/vinodhkumar232/megaminds-ids-assessment>

Video Demonstration:

https://drive.google.com/file/d/1ml-1KU25_v65fdz7EGtMdheZqS8obs_F/view?usp=sharing

Problem Overview

Modern enterprise networks generate massive volumes of continuous, high-velocity flow data. Traditional Intrusion Detection Systems (IDS), which rely predominantly on static, signature-based rulesets (e.g., standard Snort or Suricata configurations), are highly effective against known, deterministic exploit signatures. However, they consistently fail when confronted with polymorphic malware, zero-day attack vectors, or subtle volumetric campaigns that operate beneath static packet-matching thresholds.

Network security teams are increasingly shifting toward behavioral traffic analysis to catch anomalies that evade signature matching. While Machine Learning (ML) offers a powerful mechanism for detecting these behavioral deviations, deploying ML in operational environments introduces significant challenges:

1. **Extreme Class Imbalance:** Benign traffic typically accounts for over 90% of enterprise flows, while critical attacks (e.g., Infiltration, SQL injection) represent less than 0.05%, heavily skewing model performance.
2. **Alert Fatigue:** Flagging isolated anomalous packets without temporal or campaign-level context overwhelms Security Operations Center (SOC) analysts with false positives.
3. **Black-Box Inference:** Security analysts cannot act on alerts lacking context. ML models must explain *why* a classification was made to support forensic investigation.

2. Objectives

The primary objective of this assignment was to design, develop, and evaluate a working

ML-based IDS prototype capable of capturing network traffic data, extracting relevant statistical features, and accurately classifying traffic behavior.

Specific technical objectives included:

- Engineering a memory-safe, streaming ingestion engine to simulate real-time network interface monitoring.
- Implementing probabilistic confidence scoring to differentiate between high-fidelity detections and uncertain anomalies.
- Developing an explainability layer to provide analysts with mathematical justifications for flagged malicious campaigns.
- Evaluating the system against a modern, public benchmark dataset encompassing both normal baseline traffic and multi-vector attacks.

3. Technology Selection and Justification

To build a robust, scalable prototype, the following technology stack was selected:

- **Programming Language: Python 3.10+**
 - *Justification:* Python is the industry standard for both machine learning engineering and rapid cybersecurity prototyping. Its extensive ecosystem of data processing libraries ensures rapid iteration.
- **Detection Framework: scikit-learn**
 - *Justification:* Chosen for its highly optimized implementation of ensemble methods (Random Forest). Unlike deep neural network frameworks (TensorFlow/PyTorch) which require GPU acceleration and vast compute resources, scikit-learn allows for rapid training on tabular network flow data, making the solution highly portable and lightweight.
- **Data Processing Pipeline: pandas and numpy**
 - *Justification:* Essential for handling the extreme scale of the dataset (2.8 million records). pandas enables efficient memory chunking and data sanitization, while numpy provides the fast matrix operations required for real-time inference vectorization.

4. Traffic Data & Dataset Description

4.1 Dataset Selection: CIC-IDS-2017

The pipeline is trained and evaluated using the Canadian Institute for Cybersecurity **CIC-IDS-2017** benchmark dataset.

- *Justification:* Older datasets (such as KDD99 or NSL-KDD) are obsolete; they fail to represent modern attack topologies or current benign traffic distributions (e.g., heavily encrypted HTTPS/TLS traffic). CIC-IDS-2017 provides realistic B-Profile background traffic and up-to-date attack scenarios (DDoS Hulk, GoldenEye, Patator brute-force tools), making it the optimal choice for evaluating a modern ML detection engine.

4.2 Data Sanitization & Hygiene

Raw PCAP-derived flow records contain parsing anomalies that destabilize ML pipelines. A custom preprocessing engine (src/merge_datasets.py) was developed to perform the following operations across the 2,828,075 total flows:

- **Header Standardization:** Strips leading/trailing whitespace across 80+ column names.
- **Non-Finite Value Removal:** Replaces floating-point infinities (inf, -inf) resulting from zero-duration flow divisions (e.g., Flow Bytes/s) and drops null entries.
- **Label Encoding:** Maps 15 categorical string labels into deterministic numeric targets, exporting a label_encoder.pkl artifact for seamless inverse transformation during real-time inference.

5. Feature Extraction Methodology

Rather than feeding all 80+ raw statistical counters into the model—which introduces severe computational overhead, multicollinearity, and inline processing latency—the pipeline extracts a curated **14-dimensional feature vector**. These features were explicitly chosen to capture timing, volumetric symmetry, and TCP control signaling without relying on Deep Packet Inspection (DPI).

Feature Name	Network Layer / Scope	Operational & Analytical Justification
Destination Port	Transport (L4)	Identifies targeted service endpoints (e.g., Port 80/443 for HTTP/DDoS, Port 21 for FTP).
Flow Duration	Temporal / Flow	Distinguishes rapid port probes (microseconds) from long-lived exfiltration sessions.
Total Fwd / Bwd Packets	Volumetric	Measures client-to-server traffic volume and server responsiveness; asymmetrical flows indicate spoofing.
Total Length Fwd / Bwd	Payload / Byte	Captures outbound payload volume; flags volumetric payload anomalies without reading encrypted contents.

Flow Bytes/s & Packets/s	Throughput / Rate	Detects sudden high-bandwidth exfiltration or bandwidth-saturating DoS events.
Packet Length Mean / Avg	Statistical / Flow	Average packet sizes reveal protocol discrepancies (e.g., small non-payload packets in scans).
SYN / ACK Flag Counts	TCP Signaling (L4)	Abnormally high SYN counts without subsequent ACKs detect SYN flood DoS & stealth scans.
PSH / RST Flag Counts	TCP Signaling (L4)	Push/Reset flags force buffer flushing or connection aborts; common in interactive brute-force attempts.

6. Detection Methodology & System Architecture

The detection pipeline consists of two complementary layers: a Stateful Sequential Ingestion Engine and a Supervised Classifier.

6.1 Supervised Model Configuration

The core detection engine is a **Random Forest Classifier**.

- *Why Random Forest?* It is highly resistant to overfitting on noisy network data and provides intrinsic feature weighting for explainability.
- The model was configured with `n_estimators=100` (balancing stability and latency), `max_depth=20` (preventing over-memorization), and `class_weight='balanced_subsample'` (compensating for the extreme scarcity of minority attack classes).

6.2 Probabilistic Confidence Scoring

Rather than relying on deterministic binary outputs, the inference engine executes via `predict_proba()`.

- If the model's prediction confidence is $\geq 85.0\%$, the flow is flagged as **HIGH** Severity.
- If confidence falls below 85.0% , the alert is dynamically downgraded to **UNCERTAIN**

Severity. This allows analysts to filter out borderline edge cases and focus on high-fidelity threats.

6.3 Batch Ingestion & 30% Campaign Threshold

Network attacks manifest as temporally correlated streams, not single packets. The engine ingests flows in 25-flow batches using a sliding memory buffer (`collections.deque`) to maintain a baseline history of recent traffic.

A Critical Campaign Alert triggers only when anomalous flow density exceeds **30% of the active monitoring window**.

- *Threshold Justification:* Triggering alerts on a single anomalous flow generates massive False Alarm Rates due to transient network noise or benign client retries. A 30% threshold optimally suppresses transient noise (reducing alert fatigue) while guaranteeing near-100% capture rates for sustained volumetric attacks (DDoS, Brute Force).

6.4 AV-Safe Explainability Engine

When a campaign breaches the 30% threshold, the system provides forensic context. Originally designed using SHAP (SHapley Additive exPlanations) for per-flow local explainability, strict endpoint application control policies blocked the required DLL injections. To ensure the system remained highly available and secure, an AV-safe fallback was implemented. The system aggregates the model's feature importance weights specifically for the batch of malicious flows, outputting the mathematical drivers of the alert (e.g., explicitly stating that *Flow Duration* and *Destination Port* triggered the anomaly).

7. Testing, Evaluation, and Results

The pipeline was evaluated on a 30% unseen holdout test set (848,423 flows) via stratified partitioning.

7.1 Global Metrics

- **Overall Test Accuracy:** 98.41%
- **Weighted Average Precision:** 0.9958
- **Weighted Average Recall:** 0.9841
- **Weighted Average F1-Score:** 0.9895

7.2 Analysis of Minority Class Limitations

While the model achieved near-perfect F1-scores (≥ 0.98) for dominant volumetric classes (DDoS, PortScan, Benign), performance degraded on rare minority classes, resulting in a lower Macro F1-Score (0.7508).

- **Web Attacks (XSS, SQLi):** Web application attacks frequently occur over standard HTTP ports with flow statistics indistinguishable from normal user browsing (e.g., a single

malicious POST request looks statistically identical to a benign POST request). Because this flow-based engine does not utilize Deep Packet Inspection (DPI) to read L7 payloads, it struggles to isolate these attacks reliably.

8. Mandatory Demonstration Scenarios

The developed prototype successfully executes all mandatory demonstration scenarios outlined in the assessment via the `src/run_scenarios.py` CLI script:

- **Scenario 1 — Normal Traffic Baseline:** The system processes pure benign traffic streams, maintaining 99.8%–100.0% confidence with zero false alarms, establishing a reliable behavioral baseline.
- **Scenario 2 — Reconnaissance / Port Scan:** The system correctly identifies isolated scanning probe flows. However, because the scanning volume remains below the 30% campaign threshold, it suppresses the critical alert, successfully demonstrating alert fatigue reduction.
- **Scenario 3 — Denial-of-Service:** The engine detects a high-density stream of DDoS traffic, flagging it with 100% confidence. It triggers a Critical Alert and outputs the explainability metrics (highlighting abnormal Flow Packets/s).
- **Scenario 4 — Additional Attack (Brute Force):** The system flags a sustained FTP-Patator brute-force campaign targeting Port 21, correctly identifying the abnormally short flow durations and high packet rates characteristic of automated password guessing.
- **Scenario 5 — Ambiguous / Missed Case (Infiltration):** During an infiltration scenario, the model detects subtle lateral movement but assigns it a low confidence score (60.0%), correctly categorizing the severity as UNCERTAIN. This reveals the limitation of using supervised models for highly ambiguous, low-footprint attacks that closely mimic benign administrative traffic.

9. Network and Security Considerations

Designing an IDS requires balancing detection efficacy with operational realities:

- **Inline Processing Latency:** Computing statistical counters and running ML inference introduces buffer contention. An operational deployment of this pipeline must execute asynchronously (e.g., via a network tap or SPAN port mirror) to ensure it does not introduce latency or bottleneck inline network throughput.
- **Encrypted Traffic Blindness:** As enterprise networks mandate TLS 1.3 encryption, DPI is becoming obsolete. This system's reliance on flow metadata (timing, sizing, TCP flags) represents a forward-looking approach capable of evaluating encrypted traffic without breaking privacy.
- **Adversarial Evasion:** Attackers aware of behavioral ML models can introduce artificial flow jitter or fragment packets to mimic human behavior, potentially evading this specific feature set.

10. Future Improvements

To mature this prototype into a production-ready enterprise defense system, the following enhancements are proposed:

1. **Hybrid Architecture (Supervised + Unsupervised):** The current implementation is a supervised multiclass classifier limited to known signatures. Future iterations must implement an unsupervised anomaly detection layer (e.g., Isolation Forest or Autoencoders) to detect true zero-day threats by modeling purely benign baselines.
2. **Temporal Partitioning Evaluation:** The current 70/30 stratified split risks data leakage due to session-level flow correlations. A more rigorous evaluation would use temporal partitioning (e.g., training on Monday–Wednesday data, testing on Thursday–Friday data).
3. **True Local Explainability:** Integrating a securely authenticated TreeSHAP implementation would allow for true local, per-instance explainability without triggering host AV policies.

11. Conclusion & Deliverables

This project successfully delivers a working, AI-driven Network Intrusion Detection System. By pairing a highly optimized 14-feature Random Forest model with a streaming batch ingestion engine, confidence scoring, and campaign-level alert suppression, the prototype effectively bridges the gap between theoretical machine learning and operational security monitoring.

Project Deliverables:

- **GitHub Repository:** <https://github.com/vinodhkumar232/megaminds-ids-assessment>
 - *(Includes complete source code, datasets processing scripts, model artifacts, and execution instructions in the README.md).*
- **Video**
Demonstration: https://drive.google.com/file/d/1mI-1KU25_v65fdz7EGtMdheZqS8obs_F/view?usp=sharing